第20课 选择排序与插入排序

> 本讲义配合课堂例题讲解使用，代码可直接在 Dev-C++ 中运行验证。

五、课堂例题

例题 1：选择排序——从小到大排队

题目：输入 n 个整数，使用**选择排序**将它们从小到大排列并输出。

输入样例：

```
5
5 3 8 1 4
```

输出样例：

```
1 3 4 5 8
```

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int a[1005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    // 选择排序：每一轮从未排好的部分选出最小值
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;  // 先假设"未排好区"第一个就是最小的
        // 在未排好区里找真正的最小值
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[minIdx]) {
                minIdx = j;  // 发现更小的，更新下标
            }
        }
        // 把最小值交换到位置 i（已排好区的末尾）
        if (minIdx != i) {  // 如果最小值不在位置 i，才需要交换
            int temp = a[i];
            a[i] = a[minIdx];
            a[minIdx] = temp;
        }
    }

    // 输出排好的数组
    for (int i = 0; i < n; i++) {
        if (i > 0) cout << " ";
        cout << a[i];
    }
    cout << endl;

    return 0;
}
```

讲解思路：

1. 外层循环 `i` 从 0 到 `n-2`：表示当前要确定第 `i` 个位置的值（共需要 `n-1` 轮）。
2. 每轮开始，先假设 `minIdx = i`，即"未排好区第一个"是最小的。
3. 内层循环 `j` 从 `i+1` 到 `n-1`：在未排好区里找有没有更小的。如果 `a[j] < a[minIdx]`，就更新

`minIdx = j`。
4. 内层循环结束后，`minIdx` 就是真正最小值的位置。用三步交换法把 `a[i]` 和 `a[minIdx]` 交换。
5. 外层循环结束，数组就排好了！

> **类比记忆**：外层循环 = 老师叫"第几个位置站好"，内层循环 = 老师在剩下的人里扫一遍找最矮的。

例题 2：插入排序——扑克牌整理

题目：输入 n 个整数，使用**插入排序**将它们从小到大排列并输出。

输入样例：

```
5
5 3 8 1 4
```

输出样例：

```
1 3 4 5 8
```

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int a[1005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    // 插入排序：从第 2 张牌开始，每张牌插到前面合适的位置
    for (int i = 1; i < n; i++) {
        int temp = a[i];  // 把新牌抽出来放进"安全口袋"
        int j = i - 1;     // 从新牌前一个位置开始往左看
        // 往左找位置：如果左边的牌比新牌大，就往右挪一格
        while (j >= 0 && a[j] > temp) {
            a[j + 1] = a[j];  // 把大牌往右推
            j--;                // 继续往左看
        }
        a[j + 1] = temp;  // 找到合适位置，把新牌插进去！
    }

    // 输出排好的数组
    for (int i = 0; i < n; i++) {
        if (i > 0) cout << " ";
        cout << a[i];
    }
    cout << endl;

    return 0;
}
```

讲解思路：

1. 外层循环 `i` 从 1 开始（第 0 张牌不用插，它自己就是"有序的一张牌"）。
2. `temp = a[i]`：把新牌抽出来保护好。**这一步千万不能漏！**
3. `j = i - 1`：从新牌的左边第一个位置开始。
4. `while` 循环做两件事：

- 检查 `j >= 0`（还没走到数组最左边）
- 检查 `a[j] > temp`（左边的牌比新牌大）

- 两个条件都满足，就把 `a[j]` 往右挪一格（`a[j+1] = a[j]`），然后 `j--` 继续往左看。
5. `while` 结束时，`j+1` 就是新牌应该插入的位置。
6. `a[j+1] = temp`：把新牌插进去！

> **为什么是 `j+1` 而不是 `j`？** 因为 `while` 循环结束时，`j` 已经多退了一步（要么 `j < 0` 了，要么 `a[j]` 不比 `temp` 大了），所以新牌应该放在 `j` 的右边，即 `j+1`。

例题 3：三种排序都会写——排序小能手

题目：输入 n 个整数和一个数字 k（k 为 1、2 或 3）。当 k=1 时用冒泡排序，k=2 时用选择排序，k=3 时用插入排序，将数组从小到大排列并输出。

输入样例：

```
5 2
4 2 7 1 3
```

输出样例：

```
1 2 3 4 7
```

完整代码：

```
#include <iostream>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;

    int a[1005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    if (k == 1) {
        // 冒泡排序（上节课学的）
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }
    } else if (k == 2) {
        // 选择排序
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                if (a[j] < a[minIdx]) {
                    minIdx = j;
                }
            }
            if (minIdx != i) {
                int temp = a[i];
                a[i] = a[minIdx];
                a[minIdx] = temp;
            }
        }
    } else {
        // 插入排序
        for (int i = 1; i < n; i++) {
            int temp = a[i];
            int j = i - 1;
            while (j >= 0 && a[j] > temp) {
                a[j + 1] = a[j];
                j--;
```

```
            }
            a[j + 1] = temp;
        }
    }

    // 输出
    for (int i = 0; i < n; i++) {
        if (i > 0) cout << " ";
        cout << a[i];
    }
    cout << endl;

    return 0;
}
```

讲解思路：

1. 这道题是一道**三合一综合题**！三种排序方法用 `if-else` 选择。
2. 重点不是这道题本身多难，而是让同学们**把三种排序的代码都写一遍**，加深肌肉记忆。
3. 引导学生观察：三段代码虽然长得不同，但**排序结果完全一样**——条条大路通罗马！

> **成就感时刻**：同学们，你们现在已经掌握了**三种排序方法**了！在很多初中生甚至高中生还搞不清楚排序的时候，你们已经能写三种了，是不是很骄傲？

附：例题源代码与运行结果

例题 1：Selection Sort (Ascending)

```
// Input n integers, sort them ascending using selection sort.
// Core idea: each pass selects the minimum from unsorted region
// and places it at the end of the sorted region.

#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int a[1005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    // Selection sort: each pass finds the minimum in unsorted region
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;  // Assume the first unsorted element is the minimum
        // Scan the unsorted region for the real minimum
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[minIdx]) {
                minIdx = j;  // Found a smaller one, update index
            }
        }
        // Swap the minimum to position i (end of sorted region)
        if (minIdx != i) {
            int temp = a[i];
            a[i] = a[minIdx];
            a[minIdx] = temp;
        }
    }

    // Output sorted array
    for (int i = 0; i < n; i++) {
        if (i > 0) cout << " ";
        cout << a[i];
    }
    cout << endl;

    return 0;
}
```

Sample Input

```
5
5 3 8 1 4
```

Expected Output

```
1 3 4 5 8
```

例题 2：Insertion Sort (Ascending)

```
// Input n integers, sort them ascending using insertion sort.
// Core idea: like sorting playing cards - pick each card and
// insert it into the correct position among already sorted cards.

/*
#include <iostream>
using namespace std;

int main() {
    int n;
    cin >> n;

    int a[1005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    // Insertion sort: start from the 2nd element (index 1)
    for (int i = 1; i < n; i++) {
        int temp = a[i];  // Take the new card out and save it
        int j = i - 1;     // Start comparing from the card to its left
        // Shift larger cards to the right to make room
        while (j >= 0 && a[j] > temp) {
            a[j + 1] = a[j];  // Push the larger card right
            j--;                // Continue looking left
        }
        a[j + 1] = temp;  // Insert the new card at the correct position
    }

    // Output sorted array
    for (int i = 0; i < n; i++) {
        if (i > 0) cout << " ";
        cout << a[i];
    }
    cout << endl;

    return 0;
}
*/
```

Sample Input

```
5
5 3 8 1 4
```

Expected Output

```
1 3 4 5 8
```

例题 3：Three Sorting Methods in One Program

```
// Input n integers and k (1, 2, or 3).
// k=1: bubble sort, k=2: selection sort, k=3: insertion sort.

/*
#include <iostream>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;

    int a[1005];
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    if (k == 1) {
        // Bubble sort (from Lesson 19)
        for (int i = 0; i < n - 1; i++) {
            for (int j = 0; j < n - 1 - i; j++) {
                if (a[j] > a[j + 1]) {
                    int temp = a[j];
                    a[j] = a[j + 1];
                    a[j + 1] = temp;
                }
            }
        }
    } else if (k == 2) {
        // Selection sort
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                if (a[j] < a[minIdx]) {
                    minIdx = j;
                }
            }
            if (minIdx != i) {
                int temp = a[i];
                a[i] = a[minIdx];
                a[minIdx] = temp;
            }
        }
    } else {
        // Insertion sort
        for (int i = 1; i < n; i++) {
            int temp = a[i];
```

```
            int j = i - 1;
            while (j >= 0 && a[j] > temp) {
                a[j + 1] = a[j];
                j--;
            }
            a[j + 1] = temp;
        }
    }

    // Output
    for (int i = 0; i < n; i++) {
        if (i > 0) cout << " ";
        cout << a[i];
    }
    cout << endl;

    return 0;
}
*/
```

Sample Input

```
5 2
4 2 7 1 3
```

Expected Output

```
1 2 3 4 7
```